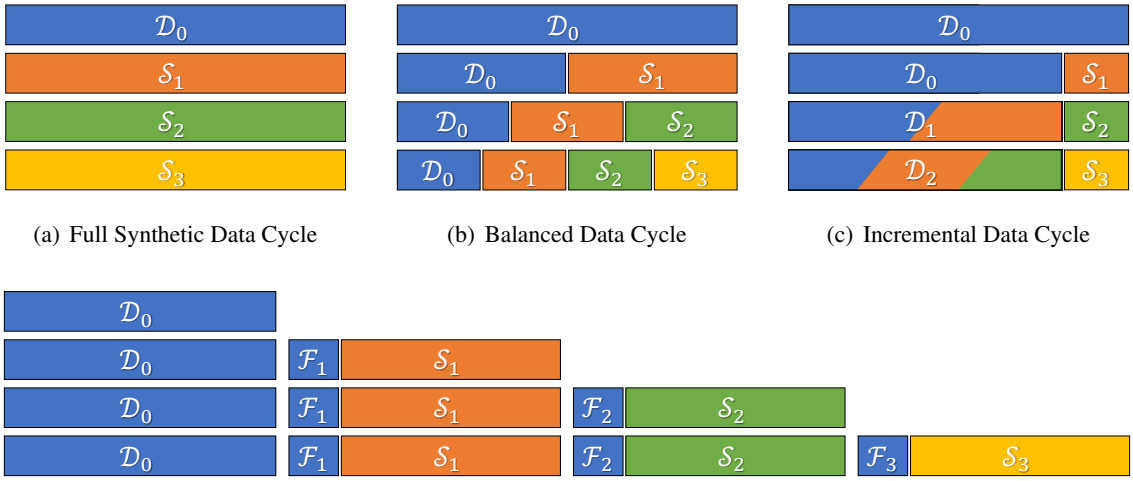


(a) Full Synthetic Data Cycle (b) Balanced Data Cycle (c) Incremental Data Cycle

(d) Expanding Data Cycle

Figure 2: Each sub figure represents a different data cycle exemplary for four generations. The first row in each sub figure is the original dataset $\mathcal{D}_0$. The second, third and fourth row depict the dataset $\mathcal{D}_1$, $\mathcal{D}_2$, and $\mathcal{D}_3$ respectively.

code those strings to a sequence of tokens (`True`, `False`, `not`, `and`, `or`, `(`, `)`, `<eos>`, where `<eos>` is a stop token and indicates the end of an expression) and use these sequences to train a language model.

3.2 Measuring the Diversity of the Model's Output

A generative model does not only need to generate correct outputs but also a diverse set of outputs. To measure the diversity of a sample from a LLM we use the Levenshtein diversity (Wittenberg et al., 2023) in our experiments. This metric calculates the pairwise Levenshtein distance between each expression in the sample normalized by the number of tokens of the longer expression of each pairwise comparison and averaged over the number of pairwise comparisons. If the average pairwise normalized Levenshtein distance is close to zero the diversity within a sample is low. A value closer to one suggests a stronger diversity within a sample. The Levenshtein distance itself is a metric for measuring the distance between two strings and is defined as the number of edits (deletion, addition, substitution) required to change one string to another (Levenshtein et al., 1966). Normally, this edit distance is calculated on a character basis, however, we calculate it on the tokens from our encoding of the logic expressions to accommodate for the different length each token has in character representation.

3.3 Data Cycles

We define a *data cycle* as the way a dataset $\mathcal{D}_t$ is constructed in generation t from the original data $\mathcal{D}_0$, potential fresh data $\mathcal{F}_t$ from the original distribution, and the generated data from the current and previous generations $\mathcal{S}_{1...t}$. This way of constructing a new dataset may vary and different data cycles are possible. In the image generation domain, previous work suggests that the self-consuming training loop is influenced by the data cycle being used (Alemohammad et al., 2023). Therefore, inspired by their work we conduct our experiments with the following four different data cycles (also depicted in Fig. 2):

Full Synthetic Data Cycle: In the most extreme case of a self-consuming training loop a new model $\mathcal{M}_t$ is only trained on the generated data from the last generation so that $\mathcal{D}_{t-1} = \mathcal{S}_{t-1}$. We call this a *full synthetic* data cycle. Normally, new datasets would still contain the original data when they are collected in practice. However, we can use this data cycle to study the most extreme changes in behavior of models from generation to generation.

Balanced Data Cycle: We refer to the second data cycle as the *balanced* data cycle. In this data cycle, we construct the new dataset $\mathcal{D}_t$ from equal parts of all the previous samples $\mathcal{S}_{1...t}$ and the original dataset $\mathcal{D}_0$ so that every previous generation contributes $m * \frac{1}{t+1}$ logic expressions to the new dataset $\mathcal{D}_t$ of size m.

Incremental Data Cycle: The third data cycle we study is called the *incremental* data cycle. In this data cycle, the new dataset is created by a portion $(1-\lambda)$ of the last dataset $\mathcal{D}_{t-1}$ and a portion λ of new sampled data $\mathcal{S}_t$ so that $\mathcal{D}_t = (1-\lambda) * \mathcal{D}_{t-1} + \lambda * \mathcal{S}_t$ while holding the size m of the dataset constant each generation. A $\lambda = 1$ would result in a full synthetic data cycle. We chose $\lambda = 0.1$ for our initial experiments and later used different values of λ to study the influence of this parameter on diversity.

Expanding Data Cycle: The first three data cycles create a dataset of equal size in each generation. In practice, however, the dataset would grow each generation by adding new generated data to the already existing dataset. Additionally, fresh data samples $\mathcal{F}_t$ from the original distribution (e.g. human generated) would also be added at each generation. Consequently, the last data cycle we study is an *expanding* data cycle. At each generation the new dataset is created so that $\mathcal{D}_t = \mathcal{D}_{t-1} + (1-\lambda) * \mathcal{F}_t + \lambda * \mathcal{S}_t$, where λ is the portion of generated data we add each generation and $(1-\lambda)$ is the portion of fresh real data added at each generation. For this data cycle, we chose $\lambda = 0.9$ for our initial experiments and later used different values of λ to study the influence of fresh data in an expanding data cycle on diversity.

3.4 Model Architecture and Training

We employ a GPT-style LLM using the open-source implementation *nanoGPT*[4] (MIT license) in our experiments. The model accepts a context of up to 256 tokens and consists of 6 attention layers with 6 attention heads each and an embedding dimensionality of 384, resulting in roughly 10.6 million parameters.

During training we use a batch size of 64 and a dropout rate of 0.2, training for 5000 iterations minimizing cross entropy loss, starting with a learning rate of 10^{-3} decaying to 10^{-4}, to achieve a sufficient fitting of the training data. We split the dataset in 90% training and 10% validation data. During training we calculate the validation error every 250 iterations and use the model parameters with the lowest validation error as our final model M_t for each generation t in the self-consuming training loop.

We use the trained model at each generation to sample $m = 10,000$ logic expressions. During sampling, we auto-regressively generate new tokens with temperature 0.8 and feed them back into the model until a stop token `<eos>` is sampled up to a maximum of 200 tokens per expression. We run the self-consuming training loop for $T = 50$ generations in each experiment.

All experiments are performed on a workstation using a NVIDIA Turing architecture graphics card.

[4] https://github.com/karpathy/nanoGPT

4 Results

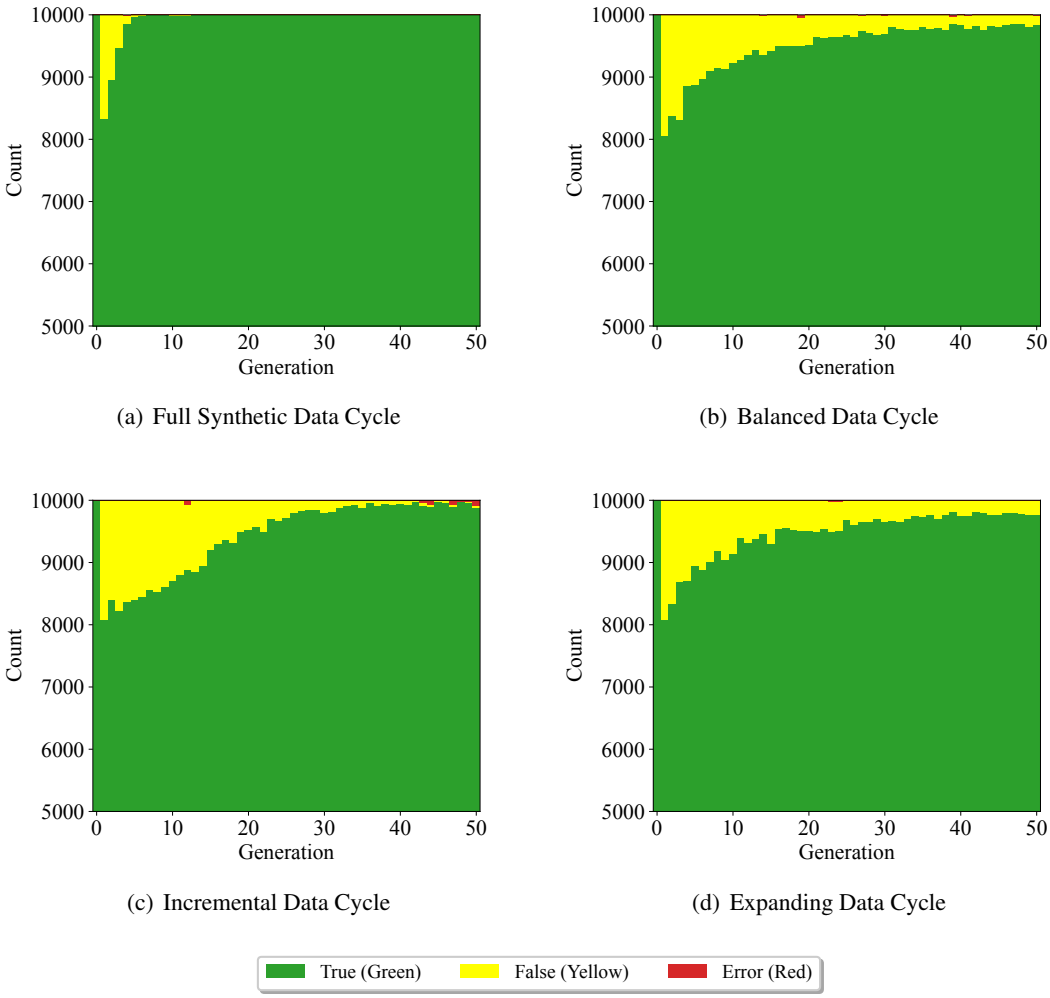
We present our experimental results. First, we describe our results on the performance in terms of correctness of models trained in a self-consuming training loop. Afterwards, we present our results in terms of diversity for those models.

4.1 Correctness of Generated Content

We first study the correctness of the expressions within a generated sample $\mathcal{S}_t$ from a model $\mathcal{M}_t$ at each generation t. As described in Sec. 3.1, we consider an expression to be syntactically correct if it can be parsed without error. Since the original dataset $\mathcal{D}_0$ only consists of `True` expressions, we consider a generated expression to be semantically correct if it also evaluates to `True`. A semantically correct expression is also syntactically correct.

Figure 3 displays the composition of samples $\mathcal{S}_t$ generated during a self-consuming training loop. Each subplot presents the results for a different data cycle: a) full synthetic, b) balanced, c) incremental, and d) expanding. Every bar in a subplot displays the composition of $\mathcal{S}_t$ generated from model $\mathcal{M}_t$ at generation t, except for the first bar in each subplot which displays $\mathcal{D}_0$. The green portion of a bar indicates the number of syntactically correct expressions that evaluate to `True` in that sample. The yellow part of a bar represents the number of syntactically correct expressions that evaluate to `False`, and the red part of a bar displays the number of syntactically incorrect expressions that result in an error when being parsed.

Overall, we see that only very few expressions are syntactically incorrect. This indicates that the models are sufficiently trained and can correctly learn the syntactic rules of a logic expression. We see a drop of around 20% in the number of semantically correct expressions from the original data $\mathcal{D}_0$ to the first sample $\mathcal{S}_1$ in every data cycle. Interestingly, the number of semantically correct expressions increases afterwards over the course of the self-consuming training loop. The speed at



(a) Full Synthetic Data Cycle

(b) Balanced Data Cycle

(c) Incremental Data Cycle

(d) Expanding Data Cycle

Figure 3: The composition of each sample $\mathcal{S}_t$ from model $\mathcal{M}_t$ at generation t (the first bar displays the composition of $\mathcal{D}_0$) with regards to the number of syntactically and semantically correct expressions. Green indicates syntactically correct expressions that evaluate to `True`, yellow indicates syntactically correct expressions that evaluate to `False`, and red indicates syntactically incorrect expressions that result in an error when being parsed. Each subplot displays the results for a different data cycle.

which this number increases depends on the data cycle. We can see the fastest increase for the full synthetic data cycle in which the whole sample consists of `True` expressions by generation $t = 6$. The incremental data cycle takes consistently longer but also nearly reaches this point by generation $t = 36$. While the balanced data cycle has an initially steeper increase in semantically correct expressions, it does not reach the point of a completely semantically correct sample by the end of 50 generations but comes very close to it. Lastly, the expanding data cycle shows this trend as well. Overall, we see that the self-consuming training loop seems to help with generating more semantically correct expressions in our experiments and the rate at which this happens differs by data cycle.

4.2 Diversity of Generated Content

While we can see an increase in correctness over generations in our experiments, contemporary work in the image generation domain suggests, that this comes with a loss of diversity (Martínez et al., 2023; Alemohammad et al., 2023; Bertrand et al., 2023). Therefore, in this section we study the diversity within a sample $\mathcal{S}_t$ of model $\mathcal{M}_t$ for each generation t.

Figure 4 displays the average pairwise normalized Levenshtein distance over generations of a self-consuming training loop for different data cycles. For each data cycle, we observe a decrease in diversity over the course of generations, with the degree of decrease varying depending on the data cycle.